

Spring Boot Hour 3 Container, Beans & Scope

Spring Boot — 8-Hour Plan · HOUR 3

Inside the Container: Beans, Wiring Conflicts, Lifecycle & Scope

Goal: Hour 1 told you "Spring creates and manages beans." This hour opens the container's lid and watches it happen — how beans get registered, what Spring does when wiring is ambiguous, what a bean's life looks like, and how many copies of a bean exist. These are the questions that separate "I've used Spring" from "I understand Spring."

3.1 Two ways a bean gets born

Everything in the container starts with one question: **how does Spring know a class should become a bean?** There are exactly two doors in.

Door 1 — Stereotype annotations (for YOUR classes)

You mark your own class, Spring's scanner finds it:

```
@Component    // the generic one – "make me a bean"
@Service       // same thing, but says "I'm business logic"
@Repository    // same thing, but says "I'm data access"
@Controller / @RestController // same thing, but says "I handle web requests"
```

You learned in Hour 2 that these are functionally almost identical. The precise picture: `@Service`, `@Repository`, `@Controller` are themselves annotated with `@Component` internally — they're *specializations*. Three reasons the separate names exist:

1. **Readability** — the annotation instantly tells a reader which layer this is.
2. **@Repository has a real superpower** — it wraps database exceptions (`SQLException` & friends, which vary per database) into Spring's uniform `DataAccessException` family. Your service layer never needs to know which DB threw what.
3. **Future tooling** — Spring can (and does) attach layer-specific behavior to specific stereotypes.

Door 2 — @Configuration + @Bean (for classes you DON'T own)

Here's the puzzle: you want a bean of some **third-party class** — say, `ObjectMapper` from the Jackson library, or a `DataSource` from a connection-pool library. You can't open Jackson's source code and slap `@Component` on it. It's not your class!

So Spring gives you a second door — a factory method:

```
@Configuration    // "this class DEFINES beans"
public class AppConfig {

    @Bean           // "the object RETURNED by this method is a bean"
    public ObjectMapper objectMapper() {
        ObjectMapper mapper = new ObjectMapper();
        mapper.registerModule(new JavaTimeModule()); // configure it your way
        return mapper;                               // ← Spring takes THIS object and manages it
    }
}
```

Analogy: hiring for your restaurant. Door 1 (`@Component`) = an employee walks in wearing your uniform — the scanner recognizes them automatically. Door 2 (`@Bean`) = you hire an outside specialist (a French pastry chef who has his own uniform) — you can't change his clothes, so instead the manager writes his name into the staff register manually. Either way, he's now on staff (in the container).

INTERVIEW CHECKPOINT — asked constantly

Q: "@Component vs @Bean?"  "@Component is class-level and picked up by component scanning — used for classes I own. @Bean is method-level inside a @Configuration class — the method constructs and returns the object, so it's the way to register third-party classes or beans needing custom construction logic."

Q: "@Controller vs @Service vs @Repository vs @Component — real difference?"  "All register beans; the specific ones are specializations of @Component expressing layer intent. @Repository additionally enables exception translation into DataAccessException; @Controller ties into MVC request mapping."

And who does the scanning?

`@ComponentScan` — which you already own without knowing it, because it's baked inside `@SpringBootApplication` (Hour 1!). Crucial detail: **it scans the package of your main class and everything BELOW it**. Put a `@Service` in a package *outside* that tree, and Spring silently never finds it — your `@Autowired` fails at startup with *"No qualifying bean of type..."* This is the #1 beginner bug, and interviewers ask it as: *"Your bean isn't being detected — what do you check first?"* → 🧠: *"Whether the class is under the main application class's package tree, since default component scan starts there."*

3.2 The wiring conflict — when TWO beans match one slot

Here's where beginners get filtered. Suppose you build a notification system:

```
public interface NotificationService { void send(String msg); }

@Service
public class EmailService implements NotificationService {
    public void send(String msg) { System.out.println("Email: " + msg); }
}

@Service
public class SmsService implements NotificationService {
    public void send(String msg) { System.out.println("SMS: " + msg); }
}
```

And somewhere:

```
@Autowired
private NotificationService notificationService;    // ...which one?!
```

Think about Spring's position. `@Autowired` works **by type** by default — "find me a bean of type `NotificationService`." But there are TWO. Spring won't guess (guessing silently would be a nightmare bug). Instead your app **fails at startup**:

```
NoUniqueBeanDefinitionException: expected single matching bean but found 2:
emailService, smsService
```

Analogy: you tell the receptionist "send in the doctor." She replies: "We have TWO doctors — Dr. Email and Dr. SMS. WHICH one?" She refuses to pick randomly — a random doctor could be the wrong one for your case.

Three ways to answer her:

Fix 1 — @Qualifier: name the one you want, at the injection point

```
@Autowired
@Qualifier("smsService")           // bean names default to the class name, camelCased
private NotificationService notificationService;
```

Fix 2 — @Primary: mark a default, at the bean itself

```
@Service
@Primary                           // "when in doubt, pick me"
public class EmailService implements NotificationService { ... }
```

Now plain `@Autowired` gets `EmailService`; anyone who wants SMS still says `@Qualifier("smsService")`. **@Qualifier at the injection point beats @Primary** — specific request overrides the default.

Fix 3 — inject ALL of them (the elegant one that impresses)

```
@Autowired
private List<NotificationService> allChannels;    // Spring injects BOTH — send everywhere!
```

💬 INTERVIEW CHECKPOINT — this exact scenario is a favorite

Q: "Two beans implement the same interface. What happens on @Autowired, and how do you fix it?" 🧠: *"Startup fails with NoUniqueBeanDefinitionException, because autowiring is by type and the type is ambiguous. I resolve it with @Qualifier to name the bean at the injection point, or @Primary on one implementation as the default — @Qualifier wins over @Primary. Injecting a List to receive all implementations is also an option."*

Q: "How does @Autowired actually resolve a bean?" 🧠: *"By type first; if multiple candidates exist, it tries to narrow by field/parameter name; still ambiguous → @Primary; an explicit @Qualifier short-circuits all of this."*

One more tool from your handbook's page 5: **@Lazy** — "don't create this bean at startup; create it on first use." Default is *eager* (all singletons built at startup — so wiring failures explode immediately at boot, which is GOOD: you find out at deploy time, not at 3 AM when a user hits the code path). **@Lazy** is for genuinely expensive, rarely-used beans.

3.3 A bean's life — from birth to funeral

Your handbook (page 4) lists 5 lifecycle steps. Let's make them mean something. When the container starts:

① Instantiate	→ Spring calls the constructor	(the object is BORN)
② Populate	→ Spring injects dependencies	(@Autowired fields filled)
③ Initialize	→ YOUR custom setup hook runs	(@PostConstruct)
④ Ready	→ bean lives in the container, serving	
⑤ Destroy	→ YOUR custom cleanup hook runs	(@PreDestroy, at shutdown)

Why do steps ③ and ⑤ exist? Think about it: what if your bean needs to **open a connection, load a cache, or start a timer** as setup? You might say "do it in the constructor!" — but at constructor time (step ①), **dependencies aren't injected yet** (that's step ②!). Anything needing an injected dependency would explode with null.

So Spring gives you a hook that runs **after** injection is complete:

```
@Service
public class CacheService {
    @Autowired
    private UserRepository repo;           // filled at step ②

    public CacheService() {
        // repo is NULL here! Constructor runs at step ① – too early.
    }

    @PostConstruct                         // runs at step ③ – repo is ready
    public void warmUp() {
        System.out.println("Preloading cache with " + repo.count() + " users");
    }

    @PreDestroy                           // runs at step ⑤ – graceful shutdown
    public void cleanup() {
        System.out.println("Flushing cache to disk before dying");
    }
}
```

Analogy: a new employee's first day. ① They walk in (constructor). ② HR hands them their laptop and ID card (dependency injection). ③ Orientation/training (@PostConstruct) — you can't train them BEFORE they have their laptop! ④ They work (ready). ⑤ Exit interview, return the laptop (@PreDestroy) before leaving the company.

(Your handbook also shows `InitializingBean / afterPropertiesSet()` and `DisposableBean` — older *interface-based* ways of doing the same two hooks. Know they exist; say that **annotations are preferred** because your class doesn't get coupled to Spring interfaces.)

🗨 INTERVIEW CHECKPOINT

Q: "Explain the bean lifecycle." 🧠: "Instantiate via constructor, populate dependencies, run initialization callbacks like `@PostConstruct`, the bean serves from the container, and destruction callbacks like `@PreDestroy` run at container shutdown."

Q: "Why @PostConstruct instead of doing setup in the constructor?" ← the deeper follow-up 🧠: "Because injection happens after construction — in the constructor, autowired fields are still null. `@PostConstruct` is the first point where the bean is fully wired."

3.4 Bean scope — how many copies exist?

Final question of the hour: when three different classes all inject `UserService` ... do they get three objects or one shared object?

Answer: ONE. The default scope is `singleton` — one instance per Spring container, shared by everyone.

This surprises people. Spring builds your `UserService` ONCE at startup, keeps it in the container, and hands the SAME object to every injection point. Why? Your service has no per-user data — it's just logic. Creating a fresh copy per use would waste memory and time for zero benefit.

When you genuinely need a fresh object per request-for-it, change the scope:

```
@Component
@Scope("prototype")           // NEW instance every time someone asks the container
```

```
public class ReportBuilder {
    private List<String> lines = new ArrayList<>();    // per-use state — must not be shared!
}
```

The full table from your handbook, with the two that matter bolded:

Scope	How many instances	Use when
singleton (default)	1 per container	stateless services — 95% of your beans
prototype	new one per request to the container	beans that carry per-use mutable state
request	1 per HTTP request	web-specific
session	1 per user session	e.g., shopping cart
application / websocket	1 per ServletContext / WS session	niche

Analogy: singleton = the one microwave in the office kitchen — everyone shares it, and that's fine because it holds no one's personal data. Prototype = disposable paper cups — everyone who asks gets a fresh one, because sharing a used cup (shared mutable state) would be... bad.

The two follow-ups that catch people

Trap 1 — "Is a singleton bean thread-safe?" NO — not automatically. One instance + many simultaneous HTTP requests = many threads inside the same object at once. You know exactly what that means from the Java threading hours: if the bean has **mutable fields**, you've built a race condition (two ATMs, one account!). The reason singletons are usually fine anyway: well-written services are **stateless** — no mutable fields, only method-local variables (which live on each thread's private stack — remember?). 🧠: *"Singleton means one instance, not thread-safe. It's safe only because we keep beans stateless; any mutable field in a singleton is shared across all request threads and needs synchronization — or better, shouldn't exist."* (Watch how your Java threading prep just cashed in inside a Spring question. Interviewers LOVE this crossover.)

Trap 2 — "What if I inject a prototype bean INTO a singleton?" Think it through: the singleton is created ONCE, so its dependencies are injected ONCE... so it receives ONE prototype instance and keeps it forever. The prototype effectively *becomes* a singleton inside it! 🧠: *"Injection happens once at singleton creation, so the singleton holds a single prototype instance forever — defeating the scope. Fixes exist (ObjectFactory/Provider injection, or asking the context per use), but the key point is knowing the trap."* Even just naming this problem scores senior points.

🔧 HOUR 3 DRILL — out loud

- Two ways to register a bean? (@Component-family scan vs @Bean method in @Configuration — own classes vs third-party)
- @Component vs @Bean? (class-level + scanned vs method-level factory)
- What extra thing does @Repository do? (exception translation → DataAccessException)
- Bean not detected — first thing you check? (is it under the main class's package tree — component scan root)
- Two implementations, one @Autowired — what happens, and the fixes? (NoUniqueBeanDefinitionException → @Qualifier / @Primary / inject List; Qualifier beats Primary)
- Bean lifecycle in 5 steps? (instantiate → populate → @PostConstruct → ready → @PreDestroy)
- Why can't setup logic live in the constructor? (dependencies not injected yet — null)
- Default scope, and what does it mean? (singleton — ONE shared instance per container)
- Is a singleton thread-safe? (No — safe only if stateless; mutable field = race condition)
- Prototype inside singleton — what's the catch? (injected once → frozen single instance)

Next → Hour 4: Configuration & Profiles — application.properties vs yml, @Value vs @ConfigurationProperties, the config loading order, and dev/test/prod profiles.